

SESSION 14

Django ORM: QuerySet Deep Dive

Topics Covered: filter(), exclude(), annotate(), select_related(), prefetch_related(), Aggregations, Q() objects

Activities: Search doctors by multiple filters, Paginate doctor listing

Task 1: Create Django model Restaurant with fields name, cuisine, rating, then use filter() to get all restaurants with cuisine='Chinese' and rating > 4.

Explanation & Approach: filter() chains multiple field lookup filters.

```
# Querying Chinese restaurants rated > 4
chinese_top = Restaurant.objects.filter(cuisine__name='Chinese', rating__gt=4.0)
```

Task 2: In Django shell, use exclude() to get all users from User model who do NOT have 'gmail.com' in email address.

Explanation & Approach: exclude() negates lookup criteria, returning non-matching instances.

```
# Excluding gmail users
non_gmail_users = User.objects.exclude(email__icontains='gmail.com')
```

Task 3: Build QuerySet using annotate() to count number of reviews for each Movie in Movie and Review setup, print each movie's name with review count.

Explanation & Approach: annotate(Count('review')) performs SQL GROUP BY aggregation.

```
from django.db.models import Count
from movies.models import Movie

movies_with_counts = Movie.objects.annotate(num_reviews=Count('reviews'))
for movie in movies_with_counts:
    print(f"Movie: {movie.title} | Total Reviews: {movie.num_reviews}")
```

Task 4: Given Product and Category model (Product has foreign key to Category), use select_related() to fetch products with categories in single query.

Explanation & Approach: select_related() performs an SQL JOIN to solve the N+1 query problem for foreign key relationships.

```
# Efficient single SQL JOIN query
products = Product.objects.select_related('category').all()
for p in products:
    print(f"Product: {p.name} | Category: {p.category.name}")
```

Task 5: Use Q() objects to filter products in 'Electronics' category OR price < 1000, paginate results to show 5 per page using Django Paginator.

Explanation & Approach: *Q() objects encapsulate complex SQL OR / AND / NOT logic; Paginator breaks querysets into dynamic pages.*

```
from django.db.models import Q
from django.core.paginator import Paginator
from products.models import Product

# Complex Q filter (Electronics OR price < 1000)
queryset = Product.objects.filter(Q(category='Electronics') | Q(price__lt=1000))

# Paginate (5 items per page)
paginator = Paginator(queryset, 5)
page_1 = paginator.get_page(1)

for product in page_1:
    print(f"{product.name} - ₹{product.price}")
```